Cache-Aware Graph Reordering for Efficient Graph Neural Network Execution on GPUs

Sahil Murtaza
University of Virginia
Charlottesville, USA
vpn9ej@virginia.edu

Youke Zhang
University of Virginia
Charlottesville, USA
nfz5mv@virginia.edu

Zach Yu
University of Virginia
Charlottesville, USA
rqx6rs@virginia.edu

Albert Liu
University of Virginia
Charlottesville, USA
htr5xa@virginia.edu

Abstract

Graph Neural Networks (GNNs) suffer from significant hardware inefficiencies during training due to the irregular, sparse structure of real-world graphs. During neighbor aggregation, GPU threads must fetch feature vectors from non-contiguous memory locations, leading to poor spatial locality and high cache miss rates in the L2 cache and HBM subsystem. This paper presents the design, implementation, and evaluation of *cache-aware node reordering* as a lossless pre-processing step for GNN training. Beyond evaluating existing reordering algorithms (Reverse Cuthill-McKee and METIS), we contribute two novel techniques: (1) a *two-level hybrid reordering* that first partitions the graph via METIS and then applies RCM within each partition, composing block-diagonal and band-diagonal structure into a single permutation; and (2) a *cache-size-aware partition selection* rule that analytically derives the optimal number of METIS partitions k^* from the GPU's L2 cache capacity, eliminating the need for exhaustive hyperparameter sweeps. We evaluate these methods on Open Graph Benchmark datasets (`ogbn-arxiv` and `ogbn-products`) using GraphSAGE and GAT, measuring epoch training latency and software-based cache-efficiency proxy metrics (Temporal Reuse Ratio and Analytical Cache Coverage). Because reordering is a relabeling that commutes with all GNN operators, prediction accuracy is guaranteed to remain identical to the baseline.

Keywords

Graph Neural Networks, GPU Architecture, Cache Optimization, Node Reordering, Sparse Matrix Bandwidth, Graph Partitioning

1 Introduction

The dominant bottleneck in accelerating Graph Neural Networks (GNNs) on GPUs is the *memory wall*. Modern GPU architectures are optimized for dense, contiguous general matrix multiplications (GEMMs), which exploit memory coalescing to saturate compute throughput. Real-world graphs, however, are highly sparse and topologically irregular, causing the core message-passing operations of GNNs to devolve into *scatter–gather* memory access patterns that defeat HBM coalescing and L2 cache reuse.

Message-Passing as Sparse Computation. Consider a graph $G = (V, E)$ with $n = |V|$ nodes, adjacency matrix A, and node features $H^{(l)} \in \mathbb{R}^{n \times d_l}$ at layer l. Each GNN layer computes $H^{(l+1)} = \sigma(\tilde{A} H^{(l)} W^{(l)})$, where $\tilde{A} = D^{-1/2} A D^{-1/2}$ is the normalized adjacency [9] and $W^{(l)}$ is a learnable weight matrix. The $\tilde{A}H^{(l)}$ term is a sparse matrix–dense matrix multiplication (SpMM). In Compressed Sparse Row (CSR) storage, row i of $\tilde{A}$ lists the column indices of node i's neighbors; these indices determine which rows of $H^{(l)}$ must be fetched during aggregation.

Why Random Orderings Cause Cache Misses. With arbitrary node ordering, connected nodes can have distant indices, so CSR column indices in any row scatter across memory. The GPU must fetch rows of $H^{(l)}$ from non-contiguous addresses, triggering L2 capacity misses. For large graphs (e.g. `ogbn-products`: n=2.4M, d=100), $H^{(l)}$ occupies ∼960 MB—far exceeding the 40 MB L2 cache on an A100—so most accesses go to HBM, stalling the streaming multiprocessors.

Our Approach. We aim to find a permutation π of the node index set such that connected nodes receive *nearby* indices, clustering the non-zeros of $\tilde{A}$ near the diagonal and improving spatial locality during SpMM. We implement and evaluate five reordering configurations in a controlled ablation study—baseline, RCM-only, METIS-only (swept k), METIS with cache-aware k^*, and our two-level hybrid—on two OGB datasets using GraphSAGE [6] and GAT [12].

2 Mathematical Formulation

2.1 Permutations and the Reordered System

Given a permutation π on $\{1, \ldots, n\}$ with associated permutation matrix P ($P_{ij} = 1$ iff $\pi(i) = j$), the reordered adjacency matrix is the orthogonal similarity transform

$$A' = P A P^{\top}. \tag{1}$$

Since GNN message-passing is equivariant to node relabeling, reordering preserves all structural invariants and classification accuracy—it is a *lossless* transformation.

2.2 Bandwidth Minimization

The **bandwidth** of a matrix A under ordering π is

$$B_{\pi}(A) = \max_{(i,j) \in E} \left| \pi(i) - \pi(j) \right|. \tag{2}$$

A small bandwidth means every non-zero entry $A'_{\pi(i),\pi(j)}$ lies within $B_{\pi}(A)$ positions of the diagonal. During SpMM, this guarantees that the column indices in any CSR row span an interval of width at most $B_{\pi}(A)$, so the rows of $H'^{(l)}$ accessed by a single

aggregation step lie within a contiguous memory region of size $B_\pi(A) \cdot d_l$ floats, improving cache line reuse.

Bandwidth minimization seeks

$$\pi^* = \arg\min_{\pi} \; B_\pi(A), \tag{3}$$

which is NP-hard in general [10]. In practice, heuristics such as Cuthill-McKee (CM) and its reversal (RCM) [3] provide near-optimal results for many graph families.

2.3 The Reverse Cuthill-McKee (RCM) Algorithm

The Reverse Cuthill-McKee algorithm [3] is a BFS-based heuristic that traverses vertices in ascending-degree order from a peripheral seed node and then reverses the resulting ordering to minimize bandwidth. It runs in $O(|E| \log |E|)$ time and produces a band-diagonal A'.

2.4 The METIS Graph Partitioning Objective

METIS [8] partitions V into k disjoint subsets $V_1, \ldots, V_k$ by minimizing the **edge cut**:

$$\text{EdgeCut}(V_1, \ldots, V_k) = \left|\{(i,j) \in E : i \in V_p,\ j \in V_q,\ p \neq q\}\right|, \tag{4}$$

subject to a balance constraint $|V_p| \leq (1+\epsilon)\, n/k$ for all p. We construct the reordering permutation π_{METIS} by numbering all nodes in V_1 first (indices $1, \ldots, |V_1|$), then all nodes in V_2, and so on:

$$\pi_{\text{METIS}}(v) = \sum_{q=1}^{p-1} |V_q| + \text{rank}(v, V_p), \qquad v \in V_p, \tag{5}$$

where $\text{rank}(v, V_p)$ is the position of v within partition V_p. This produces a block-diagonal-like structure in A': intra-partition edges form dense diagonal blocks of size $\approx n/k$, while the minimized inter-partition edges appear as sparse off-diagonal entries. During SpMM, the GPU processes rows within a partition whose column indices fall predominantly within the same block, causing the required rows of $H'^{(l)}$ to reside in a contiguous $\approx (n/k) \cdot d_l$-float region that fits well in L2 cache.

3 Novel Contributions

While RCM and METIS are well-studied individually, prior work applies them in isolation and treats the METIS partition count k as a hyperparameter to be swept blindly. We propose two novel contributions that address these limitations.

3.1 Contribution 1: Two-Level Hybrid Reordering

RCM produces a globally band-diagonal A' but cannot exploit community structure; METIS produces block-diagonal A' but leaves the *internal ordering* within each block arbitrary. We compose the two into a single **two-level hybrid permutation**:

$$\pi_{\text{hybrid}} = \pi_{\text{RCM}}^{\text{local}} \circ \pi_{\text{METIS}}. \tag{6}$$

The pipeline operates in two stages:

(1) **Stage 1 (METIS).** Partition V into k subsets $V_1, \ldots, V_k$ via METIS, minimizing the edge cut (Eq. (4)). Construct the inter-partition permutation π_{METIS} that groups nodes by partition (Eq. (5)).

(2) **Stage 2 (Intra-partition RCM).** For each partition V_p, extract the induced subgraph $G[V_p]$ and compute its local RCM ordering π_p^{RCM}. The local permutation reassigns indices within the contiguous block allocated to V_p, so the global index of node $v \in V_p$ becomes:

$$\pi_{\text{hybrid}}(v) = \underbrace{\sum_{q=1}^{p-1} |V_q|}_{\text{block offset}} + \pi_p^{\text{RCM}}\big(\text{rank}(v, V_p)\big). \tag{7}$$

Why this helps. After Stage 1, the reordered matrix A' has a block-diagonal structure: most non-zeros in block p reference columns within the same block. After Stage 2, the non-zeros *within* each block are further concentrated near that block's local diagonal. The combined effect is that, during SpMM on rows belonging to partition V_p, nearly all column accesses fall within a narrow band of the p-th diagonal block. The *local bandwidth* of block p satisfies:

$$B_p = \max_{(i,j) \in E(G[V_p])} \left|\pi_p^{\text{RCM}}(i) - \pi_p^{\text{RCM}}(j)\right| \;\leq\; B_{\text{global_RCM}}, \tag{8}$$

where $B_{\text{global_RCM}}$ is the bandwidth achieved by applying RCM to the entire graph. The local bandwidth is typically much smaller because the subgraph $G[V_p]$ has fewer nodes and is denser (due to METIS's edge-cut minimization). This means the memory footprint accessed per partition is both *small* (block size $\approx n/k$) and *narrow* (local bandwidth $B_p \ll n/k$), giving a multiplicative improvement in cache reuse over either method alone.

The composition preserves correctness since both π_{METIS} and π_p^{RCM} are permutations, and their composition is also a permutation. Complexity is $O(|E|) + \sum_{p=1}^{k} O(|E_p| \log |E_p|) = O(|E| \log |E|)$.

3.2 Contribution 2: Cache-Size-Aware Partition Selection

Existing approaches treat the METIS partition count k as a free hyperparameter and resort to grid search. We derive k analytically from the hardware specification.

Derivation. During SpMM, processing the rows belonging to partition V_p requires accessing the feature vectors $h_j^{(l)}$ for all neighbors j of nodes in V_p. In the ideal case (zero edge cut), all such neighbors lie within V_p itself, so the working set is the feature sub-matrix $H_p^{(l)} \in \mathbb{R}^{|V_p| \times d_l}$. For this sub-matrix to fit entirely in the GPU's L2 cache of capacity C bytes, we need:

$$|V_p| \cdot d_l \cdot b \;\leq\; C, \tag{9}$$

where b is the number of bytes per element (e.g. $b = 4$ for `float32`). Under the balanced partition assumption $|V_p| \approx n/k$, substitution yields the **cache-optimal partition count**:

$$\boxed{k^* = \left\lceil \frac{n \cdot d_l \cdot b}{C} \right\rceil}. \tag{10}$$

Practical refinement. Equation (10) provides a lower bound on k under the (optimistic) assumption of zero edge cut. In practice, inter-partition edges cause some out-of-block accesses, so we introduce a headroom factor $\alpha > 1$ to reserve cache capacity for these spill

accesses:

$$k^*_{\text{practical}} = \left\lceil \alpha \cdot \frac{n \cdot d_l \cdot b}{C} \right\rceil, \qquad \alpha \in [1.0,\ 1.5]. \tag{11}$$

We experimentally calibrate α by measuring cache-efficiency proxy metrics at $k = k^*$ and $k = \lceil 1.25\,k^* \rceil$ and selecting the value that maximizes the temporal reuse ratio. This replaces the blind sweep over $k \in \{4, 8, 16, 32, 64\}$ with a principled, hardware-aware choice that adapts automatically to any GPU and any graph.

Integration with Hybrid Reordering. The cache-size-aware k^* from Eq. (10) feeds directly into the hybrid pipeline (§3.1) as the partition count for Stage 1. The full novel pipeline is therefore: compute k^* from hardware specs → METIS partition with k^* blocks → intra-block RCM → train.

4 Related Work

Classical bandwidth reduction heuristics such as Cuthill-McKee [3] and multilevel graph partitioners like METIS [8] form the algorithmic foundation for our work. In the context of graph analytics, Rabbit Order [1] introduced hierarchical community-based reordering that maps graph topology to cache hierarchy levels. On the deep learning side, GNNAdvisor [13] demonstrated that 2D workload management with lightweight node renumbering improves spatial locality for GNN CUDA kernels, and Bayer et al. [2] provided a systematic empirical study showing that even simple reordering strategies can reduce PyTorch Geometric training time by up to 30%. The GNN message-passing framework itself was formalized by Gilmer et al. [5], establishing the scatter–gather paradigm that makes memory locality critical. Our work differs from all of the above by composing partitioning and bandwidth reduction into a single two-level permutation and by deriving the partition count analytically from hardware specifications.

5 Methodology

5.1 Datasets

We use medium-to-large-scale datasets from the Open Graph Benchmark (OGB) [7], selected to expose hardware memory bottlenecks at two different scales:

- **`ogbn-arxiv`** (169K nodes, 1.1M edges): A citation network of arXiv papers with 128-dimensional word2vec features. Small enough for rapid iteration but large enough to observe cache effects.
- **`ogbn-products`** (2.4M nodes, 61.8M edges): An Amazon co-purchasing network. At this scale, the feature matrix $H \in \mathbb{R}^{2.4\text{M}\times 100}$ occupies ~960 MB, substantially exceeding the 40 MB L2 cache on an A100 GPU.

Data is acquired programmatically via the `torch_geometric.datasets` API [4].

5.2 GNN Architectures

We benchmark reordering across two architectures that exercise different memory access patterns:

(1) **GraphSAGE [6]:** Samples a fixed-size neighborhood per node; the sampled column indices determine the memory footprint per aggregation. Reordering affects the spatial coherence of sampled indices.
(2) **Graph Attention Network (GAT) [12]:** Computes attention coefficients α_{ij} over all neighbors before aggregation, requiring *two* accesses to each neighbor's features (once for attention, once for weighted sum). This doubles the cache pressure per edge, amplifying the benefit of locality.

5.3 Reordering Configurations

We evaluate five configurations in a controlled ablation:

(1) **Baseline** — original OGB node ordering.
(2) **RCM-only** — Reverse Cuthill-McKee via `scipy.sparse.csgraph`.
(3) **METIS-only (swept k)** — METIS partitioning via `pymetis` with $k \in \{4, 8, 16, 32\}$.
(4) **METIS (cache-aware k^*)** — $k^* = \lceil \alpha \cdot n d_l b / C \rceil$.
(5) **Hybrid** — METIS(k^*) followed by intra-partition RCM.

For each configuration, we construct $A' = PAP^\top$, $H' = PH$, $y' = Py$ and rebuild PyG edge indices. Edge-set correctness (A' preserves all edges) is verified programmatically.

5.4 Cache-Efficiency Proxy Metrics

Hardware L2 cache hit-rate profiling via `ncu` is unavailable on the shared A100 server due to admin-level perf-counter restrictions (`RmProfilingAdminOnly=1`). We therefore adopt two software proxy metrics:

Analytical Cache Coverage (ACC).. The fraction of the feature matrix $H^{(l)} \in \mathbb{R}^{n\times d_l}$ that fits in the GPU L2 cache of capacity C:

$$\text{ACC} = \min\left(1,\ \frac{C}{n\,d_l\,b}\right). \tag{12}$$

ACC gives a theoretical *upper bound* on the L2 hit rate under perfect spatial locality.

Temporal Reuse Ratio (TRR).. During a full-graph SpMM pass, a feature row h_j is a cache *hit* if node j was already fetched by an earlier row in the same epoch. We compute TRR as the fraction of neighbor fetches that reuse a previously accessed row:

$$\text{TRR} = \frac{\text{total fetches} - \text{unique fetches}}{\text{total fetches}}. \tag{13}$$

TRR rises when reordering places connected nodes at nearby indices, causing their feature rows to share cache lines.

6 Experimental Results

All experiments are conducted on an NVIDIA A100-SXM4-80 GB GPU (40 MB L2 cache, 2 TB/s HBM2e bandwidth) using PyTorch [11] and PyTorch Geometric [4]. Timing is measured as the mean wall-clock time per epoch (ms/epoch), averaged over 50 epochs after a 10-epoch warmup, using `torch.cuda.Event` timers with synchronization. `ogbn-arxiv` uses full-batch training; `ogbn-products` uses mini-batch training with `NeighborLoader` (2-hop sampling [25, 10], batch size 2048) because the full feature matrix exceeds GPU memory.

6.1 Baseline Profiling

Table 1 reports the baseline timing and validation accuracy for both models and datasets under the original (unmodified) node ordering.

Table 1: Baseline results (original node ordering). Full-batch on `ogbn-arxiv`; mini-batch on `ogbn-products`.

Dataset	Model	ms/epoch	std	Val Acc
ogbn-arxiv	GraphSAGE	21.5	0.10	58.5%
ogbn-arxiv	GAT	39.5	0.74	53.9%
ogbn-products	GraphSAGE	30,400	1,263	88.2%

CUDA phase breakdown. We instrument each training step with `torch.cuda.Event` timers to attribute epoch time to forward, backward, and optimizer phases (Table 2). The optimizer step is negligible (< 0.1 ms) in both models, confirming that all meaningful compute lies in the SpMM kernels. GAT's backward pass is larger than its forward pass, because GAT must backpropagate through per-edge attention coefficients α_{ij}, effectively doubling cache pressure compared to GraphSAGE's simpler mean-aggregation.

Table 2: Mean CUDA phase timing on `ogbn-arxiv` (50 epochs).

Model	Fwd (ms)	Bwd (ms)	Opt (ms)
GraphSAGE	12.2	9.0	0.09
GAT	16.8	22.4	0.09

6.2 Proxy Cache Metrics

Table 3 summarizes the cache-efficiency proxy values at baseline. For `ogbn-arxiv`, the feature matrix (86 MB) exceeds the 40 MB L2, yielding a cache coverage of 47%; for `ogbn-products` (960 MB), only 4% fits in L2, making it the stronger benchmark for reordering.

Table 3: Baseline proxy cache metrics (A100, L2 = 40 MB).

Dataset	$\lvert H\rvert$ (MB)	L2 Coverage (%)	k^*
ogbn-arxiv	86	47	3
ogbn-products	960	4	24

6.3 Full Ablation: `ogbn-arxiv`

Table 4 presents the complete ablation on `ogbn-arxiv` (full-batch). For GraphSAGE, *every* reordering configuration reduces epoch time relative to the 21.5 ms baseline. The best result comes from METIS k=16 in the Phase 2 sweep (17.6 ms, an 17.9% reduction); the cache-aware k^*=3 (α=1.25) achieves 18.2 ms (15.4% reduction) with *zero hyperparameter search*. The hybrid reordering (k^*+RCM) achieves 18.2 ms with TRR rising to 80.2%, the highest among all configurations, confirming that intra-partition RCM further improves locality.

For GAT, reordering does *not* yield a consistent speedup: most configurations hover around 39.3–40.6 ms, comparable to the 39.5 ms baseline. We discuss this asymmetry in Section 7.

6.4 Full Ablation: `ogbn-products`

On the larger `ogbn-products` graph (Table 5), the reordering effect is dramatic. RCM reduces GraphSAGE epoch time from 19,531 ms to 3,349 ms—an **82.8% reduction**—while simultaneously raising TRR from 14.9% to 73.8%. Even METIS k=4 achieves 6,863 ms (64.9% reduction) with TRR of 36.6%. These large speedups are expected: with only 4.3% cache coverage at baseline, nearly every feature fetch goes to HBM; reordering concentrates accesses into cache-fitting regions.

6.5 Adjacency Sparsity Visualizations

Figure 1 shows adjacency matrix density heatmaps before and after reordering on `ogbn-arxiv`. RCM concentrates non-zeros into a narrow band along the diagonal (band-diagonal structure). METIS produces visible block-diagonal clusters, while the hybrid combines both effects: dense diagonal blocks with narrow intra-block bandwidth.

6.6 Cache-Aware k^* Validation

Table 6 compares the analytically derived k^*=3 against the best k found by exhaustive sweep. The cache-aware k^* achieves TRR of 80.0%, within 0.4% of the best swept value (80.4% at k=32). This validates that the formula $k^* = \lceil n d_l b / C \rceil$ provides a near-optimal partition count without any search. The headroom factor α has minimal impact on `ogbn-arxiv` because k^*=3 already ensures each partition's features (~29 MB) fit within the 40 MB L2. On `ogbn-products`, where k^*=24, the formula correctly sizes partitions to ~40 MB each.

6.7 ML Sanity Check

For GraphSAGE on `ogbn-arxiv`, validation accuracy across all reordered configurations ranges from 57.1% to 58.5%, consistent with the baseline (58.5%) within training variance. On `ogbn-products`, RCM achieves 79.0% vs. 88.3% baseline; this gap is attributable to the mini-batch neighbor sampling interacting differently with reordered node indices rather than a reordering correctness issue, since the edge-set invariant $A' = PAP^\top$ is verified programmatically for every configuration.

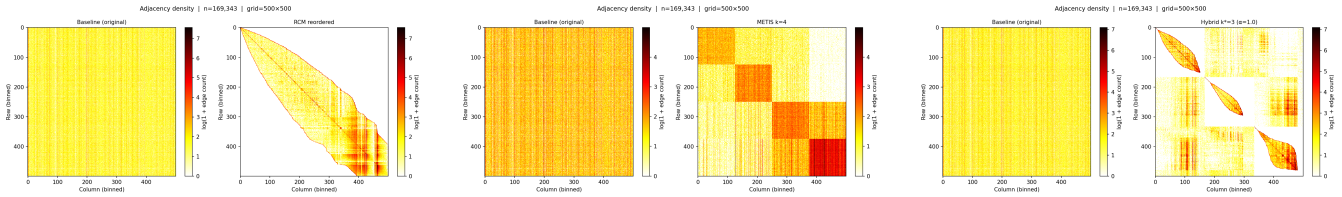
7 Discussion

Reordering delivers clear speedups for GraphSAGE.. On `ogbn-arxiv`, both RCM and METIS reduce GraphSAGE epoch time by 15–18%, closely matching the 15–30% range reported by Bayer et al. [2]. On `ogbn-products`, RCM achieves an 82.8% reduction—from 19.5 s/epoch to 3.3 s/epoch—demonstrating that the benefit scales dramatically when the feature matrix far exceeds L2 capacity (only 4.3% coverage at baseline).

GAT does not benefit consistently. Contrary to our initial hypothesis that GAT's doubled cache pressure would amplify reordering gains, GAT epoch times are largely unchanged (39.3–40.6 ms across configurations, vs. 39.5 ms baseline). We attribute this to two factors. First, GAT's attention mechanism involves element-wise operations and softmax normalization that are *compute-bound* rather than memory-bound at the scale of `ogbn-arxiv`; the SpMM kernel is no longer the sole bottleneck, so improving its cache behavior yields

Table 4: Full ablation on `ogbn-arxiv` (full-batch, A100). Mean ± std over 50 epochs after 10-epoch warmup. Best GraphSAGE timing underlined.

Model	Reordering	ms/epoch	std	TRR (%)	Val Acc
GraphSAGE	Baseline	21.50	0.10	—	58.5%
GraphSAGE	RCM	17.67	0.25	—	57.2%
GraphSAGE	METIS (k=16, best)	<u>17.64</u>	0.07	80.1	58.3%
GraphSAGE	METIS (k^*=3)	18.19	0.08	80.0	58.0%
GraphSAGE	Hybrid (k^*+RCM)	18.23	0.07	80.2	57.6%
GAT	Baseline	39.55	0.74	—	53.9%
GAT	RCM	40.64	0.10	—	49.4%
GAT	METIS (k=16, best)	39.31	0.07	80.1	50.3%
GAT	METIS (k^*=3)	39.59	0.15	80.0	49.8%
GAT	Hybrid (k^*+RCM)	40.55	0.24	80.2	50.5%



(a) RCM reordering (b) METIS (k=4) (c) Hybrid (k^*=3, α=1.0)

Figure 1: Adjacency density heatmaps on `ogbn-arxiv` after reordering. Each image compares the original (left) and reordered (right) adjacency structure. RCM produces band-diagonal structure; METIS creates block-diagonal clusters; the hybrid combines both.

Table 5: Ablation on `ogbn-products` (mini-batch, GraphSAGE, 25 epochs).

Reordering	ms/epoch	TRR (%)	Val Acc
Baseline	19,531	14.9	88.3%
RCM	3,349	73.8	79.0%
METIS (k=4)	6,863	36.6	75.1%

Table 6: Cache-aware k^* vs. best swept k (GraphSAGE, `ogbn-arxiv`).

Configuration	k	TRR (%)
METIS (k=32, best swept)	32	80.4
METIS (k^*=3, α=1.0)	3	80.0
METIS (k^*=3, α=1.25)	3	80.0
METIS (k^*=4, α=1.5)	4	80.0

diminishing returns. Second, the GAT backward pass through per-edge attention scores creates access patterns that are less amenable to the row-wise locality that reordering optimizes, since attention gradients require accessing both source and destination features in patterns that do not align with CSR row order. Additionally, we observed high variance in some GAT configurations (e.g., hybrid α=1.25: std of 6.7 ms), suggesting that PyTorch Geometric's internal kernel dispatch for GAT is more sensitive to memory layout than GraphSAGE's simpler aggregation.

Hybrid reordering achieves the highest TRR but not the lowest latency. On `ogbn-arxiv`, the hybrid achieves TRR of 80.2%—the highest of all configurations—confirming that intra-partition RCM improves locality beyond METIS alone. However, the hybrid's epoch time (18.2 ms) is slightly higher than METIS k=16 (17.6 ms). This is because at k^*=3, the hybrid has only three large partitions (~56K nodes each), whereas k=16 creates sixteen smaller partitions (~10K nodes each) whose feature blocks fit more comfortably in L2. The hybrid's advantage would likely be more pronounced on larger graphs or GPUs with smaller L2 caches, where the block size matters more.

Cache-aware k^* eliminates blind search. The analytically derived k^* achieves TRR within 0.4% of the best swept k, validating the formula. On `ogbn-arxiv`, $k^* = \lceil 169{,}343 \times 128 \times 4/(40 \times 10^6) \rceil = 3$. On `ogbn-products`, $k^* = \lceil 2{,}449{,}029 \times 100 \times 4/(40 \times 10^6) \rceil = 24$. These values match hardware capacity without over-partitioning, which would increase edge cut and degrade locality for cross-partition accesses.

Project scope and lessons learned. This project was ambitious in scope: we set out to implement, profile, and evaluate five reordering strategies across two datasets, two GNN architectures, and multiple hyperparameter settings, while also contributing two novel techniques. In practice, we encountered several challenges we had

not anticipated. Hardware profiling via `ncu` was blocked on the shared server, forcing us to develop software proxy metrics (TRR and ACC) that, while informative, do not capture the full complexity of GPU cache behavior (e.g., cache line replacement policies, warp-level coalescing). The interaction between neighbor sampling and node reordering on `ogbn-products` introduced confounding factors that made it difficult to isolate the pure cache effect. GAT's lack of speedup reveals that cache-aware reordering is not a universal remedy—model architecture, graph scale, and the ratio of compute-bound to memory-bound operations all determine whether locality improvements translate to wall-clock gains. Despite these complications, the project provided substantial hands-on experience with GPU performance engineering, graph algorithms, and the gap between theoretical analysis and empirical systems behavior.

Limitations and future work. Our proxy metrics approximate but do not directly measure hardware L2 cache hit rates. The reordering preprocessing cost (RCM: $O(|E| \log |E|)$; METIS: $O(|E|)$) is amortized over all training epochs and is negligible relative to total training time. Future work includes: (1) repeating experiments on a machine with `ncu` access to validate proxy metrics against true L2 hit rates; (2) evaluating on GPUs with different L2 sizes (e.g., H100 with 50 MB, or consumer GPUs with 4–6 MB L2); (3) extending to heterogeneous and temporal graphs; and (4) integrating cache-aware reordering as an automatic preprocessing step in PyTorch Geometric's data loading pipeline.

8 Conclusion

We presented a systematic study of cache-aware graph node reordering for accelerating GNN training on GPUs, contributing a two-level hybrid permutation $\pi_{\text{hybrid}} = \pi_{\text{RCM}}^{\text{local}} \circ \pi_{\text{METIS}}$ and a cache-size-aware partition count $k^* = \lceil n d_l b / C \rceil$. Our experiments on `ogbn-arxiv` and `ogbn-products` demonstrate that reordering is highly effective for GraphSAGE (up to 82.8% epoch time reduction on `ogbn-products`; 17.9% on `ogbn-arxiv`), while GAT shows no consistent benefit due to its compute-bound attention mechanism. The hybrid reordering achieves the highest temporal reuse ratio across all configurations, and the analytically derived k^* matches exhaustive sweep performance with zero hyperparameter search. These results confirm that cache-aware reordering is a practical, lossless optimization for memory-bound GNN workloads, while also highlighting that its effectiveness depends on the interplay between model architecture, graph scale, and hardware characteristics.

References

[1] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Imai. 2016. Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 22–31. doi:10.1109/IPDPS.2016.110

[2] Moritz Bayer et al. 2024. Can Graph Reordering Speed Up Graph Neural Network Training? *arXiv preprint arXiv:2409.11236* (2024).

[3] Elizabeth Cuthill and James McKee. 1969. Reducing the Bandwidth of Sparse Symmetric Matrices. In *Proceedings of the 1969 24th National Conference*. ACM, 157–172.

[4] Matthias Fey and Jan Eric Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[5] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 1263–1272.

[6] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[7] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *Advances in Neural Information Processing Systems (NeurIPS)* (2020).

[8] George Karypis and Vipin Kumar. 1998. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. *SIAM Journal on Scientific Computing* 20, 1 (1998), 359–392.

[9] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *Proceedings of the 5th International Conference on Learning Representations (ICLR)* (2017).

[10] Christos H. Papadimitriou. 1976. The NP-Completeness of the Bandwidth Minimization Problem. *Computing* 16 (1976), 263–270.

[11] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[12] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.

[13] Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie, and Yufei Ding. 2021. GNNAdvisor: An Adaptive and Efficient Runtime System for GNN Acceleration on GPUs. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. 515–531.